

BRENO VARGAS PEREIRA - RA: 823134327

SABRINA AMORIM - RA: 82329546

NATAN CARDOSO DE OLIVEIRA - RA: 823120309

JULIA FREITAS SANTOS - RA: 823222304

GABRIEL BALDINI - RA: 823115970

GIOVANNI LOURENÇO PASCHOIM - RA: 824215509

**FUNDAMENTOS DA TRADUÇÃO DE CÓDIGO-FONTE:
ESTUDO SOBRE ANALISADORES LÉXICOS**

1 INTRODUÇÃO: COMPILAÇÃO E INTERPRETAÇÃO

A execução de um programa escrito em uma linguagem de programação de alto nível exige sua prévia conversão para a linguagem de máquina compreendida pelo hardware. Esse processo de tradução ocorre primordialmente sob dois paradigmas: a compilação e a interpretação. Na compilação, o código-fonte é processado e traduzido em sua totalidade antes de qualquer execução, resultando na geração de um arquivo executável otimizado. Este método garante alto desempenho em tempo de execução, embora exija um tempo prévio de processamento.

Por outro lado, no paradigma da interpretação, o código-fonte é lido e executado de forma sequencial, linha por linha. O interpretador traduz e executa cada instrução de forma dinâmica, o que facilita o rastreamento e correção de erros em tempo real, porém introduz uma latência sistêmica durante a execução. Independentemente das diferenças operacionais e arquitetônicas entre linguagens compiladas e interpretadas, ambas necessitam de uma etapa inicial idêntica e fundamental para processar o texto do programa, denominada análise léxica.

2 A ANÁLISE LÉXICA

A análise léxica atua como a primeira fase formal do processo de compilação ou de interpretação. O objetivo principal deste componente é realizar a leitura estruturada dos caracteres contidos no código-fonte e agrupá-los em unidades lógicas com significado sintático, denominadas *tokens*. Cada *token* emitido representa uma estrutura válida pertencente à linguagem, como palavras reservadas, operadores lógicos ou aritméticos, delimitadores e identificadores.

Além da responsabilidade de categorização através de *tokens*, o analisador léxico atua como um filtro saneador. Ele executa a remoção de elementos que não apresentam relevância sintática ou semântica para a máquina, tais como espaços em branco, tabulações, quebras de linha e comentários inseridos pelos desenvolvedores. Dessa forma, a análise léxica assegura que as fases subsequentes recebam um fluxo de dados purificado e rigorosamente catalogado.

3 MECANISMOS FORMAIS ENVOLVIDOS

A correta delimitação e identificação dos *tokens* fundamenta-se na aplicação rigorosa de conceitos da teoria das linguagens formais e dos autômatos. O processamento baseia-se na sinergia entre expressões regulares e autômatos finitos.

3.1 Expressões Regulares

As expressões regulares consistem em notações matemáticas utilizadas para descrever declarativamente os padrões textuais aceitos por uma determinada linguagem. Através delas, é possível especificar, com precisão, as regras de composição de cada token. Por exemplo, uma expressão regular pode estabelecer que um identificador válido deve obrigatoriamente iniciar com um caractere alfabético, sendo opcionalmente seguido por uma combinação de caracteres alfanuméricos.

3.2 Autômatos Finitos

Enquanto as expressões regulares fornecem as regras lógicas, os autômatos finitos representam a implementação computacional imperativa responsável por validar as sequências de caracteres de acordo com tais modelos. Durante a análise, o autômato finito avança sequencialmente pela leitura do código e transita entre diversos estados internos lógicos. Ao alcançar uma condição de aceitação — como a detecção de um espaço delimitando uma variável — o analisador conclui o lexema lido, emite o *token* correspondente e reposiciona seu estado inicial para prosseguir com a leitura subsequente.

4 A TABELA DE SÍMBOLOS

A Tabela de Símbolos consiste em uma estrutura de dados fundamental (frequentemente implementada sob o modelo de dispersão ou *hash tables*) gerida de forma global pelo compilador ou interpretador. Sua finalidade é armazenar e indexar informações críticas referentes aos identificadores declarados no programa, como o nome de variáveis, funções e classes.

A interação de abertura com a Tabela de Símbolos ocorre de forma intrínseca durante a análise léxica. Ao detectar um *token* correspondente a um identificador inédito no código, o analisador léxico prontamente cadastra tal lexema como uma nova entrada na tabela. Neste estágio inicial, a tabela registra primariamente a representação textual do identificador. À medida que o processo avança pelas fases de Análise Sintática e Semântica, essas mesmas entradas serão recuperadas e enriquecidas com atributos vitais para o escopo lógico do programa, como tipagem de dados (inteiro, ponto flutuante), abrangência e o referencial de memória alocada para a respectiva execução física.

REFERÊNCIAS

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO (UFRJ). **Análise Léxica - Compiladores**. Notas de Aula. Disponível em: <<https://ic.ufrj.br/~fabiom/comp/02AnaliseLexicaPt1.pdf>>.

UNIVERSIDADE ESTADUAL DE CAMPINAS (UNICAMP). **Análise Léxica**. Instituto de Computação. Disponível em: <<https://www.ic.unicamp.br/~sandro/cursos/mo403/slides/1-AnaliseLexica.pdf>>.

UNIVERSIDADE FEDERAL DE MATO GROSSO DO SUL (UFMS). **Compiladores: Capítulo 2 - Análise Léxica**. Faculdade de Computação. Disponível em: <http://www.facom.ufms.br/~ricardo/Courses/CompilerI-2009/Materials/Lexical_Analisys.pdf>.

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL (UFRGS). **Compiladores: Análise lexical**. Instituto de Informática. Disponível em: <<https://www.inf.ufrgs.br/~nicolas/pdf/Compiladores02-er.pdf>>.

UNIVERSIDADE FEDERAL DE MATO GROSSO DO SUL (UFMS). **Compiladores I: Geradores de Analisadores Léxicos – Lex / Flex**. Faculdade de Computação. Disponível em: <<https://www.facom.ufms.br/~ricardo/Courses/CompilerI/Material/aula-lex-flex.pdf>>.